ANHANGUERA

ENGENHARIA DE SOFTWARE

GABRIEL DAS CHAGAS SILVA

Sistema de gerenciamento de livros

Linguagem de Programação

Unidade 2 - Explorando recursos do Python | Aula 4 - Bibliotecas e módulos

2026

GABRIEL DAS CHAGAS SILVA

Sistema de gerenciamento de livros

Relatório de aula prática apresentado à Anhanguera, no curso de Engenharia de Software, para a disciplina Linguagem de Programação.

Aluno: Gabriel das Chagas Silva
Disciplina: Linguagem de Programação
Unidade 2 - Explorando recursos do Python | Aula 4 - Bibliotecas e módulos
Data: 13 de outubro de 2026

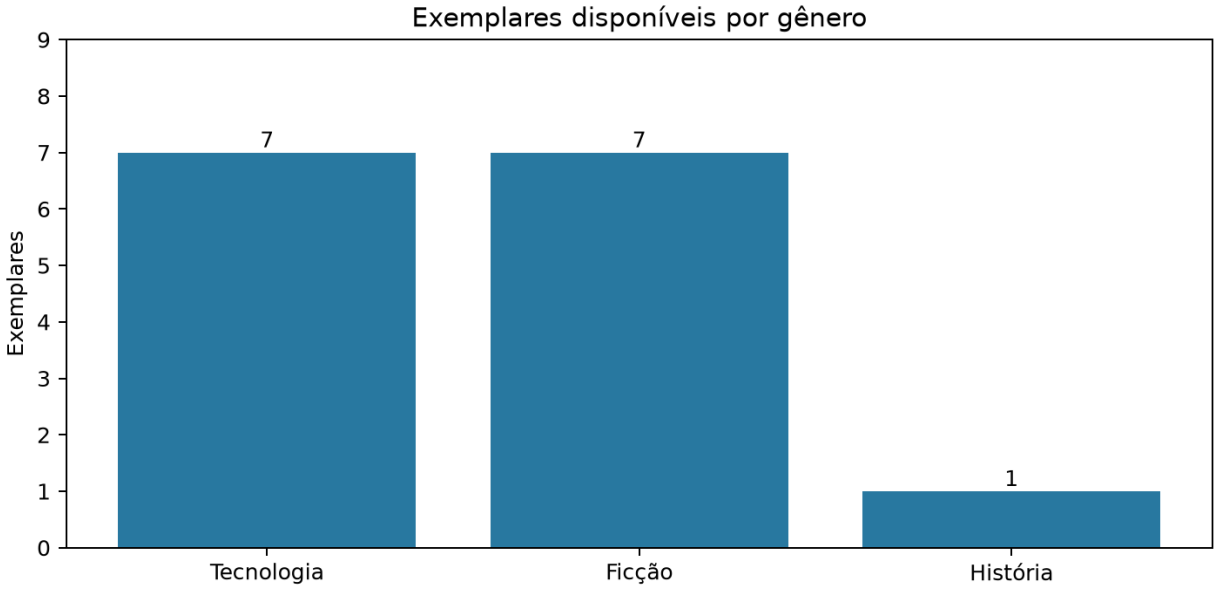
Ambiente de execução

Os códigos e testes desta entrega foram executados localmente em Python. Os notebooks .ipynb incluídos são compatíveis com o Google Colab e contêm as saídas reais da execução. As capturas mostram o notebook executado localmente; não representam uma sessão no serviço Colab.

Implementação e resultados

A classe Livro reúne título, autor, gênero e quantidade. A lista livros começa vazia. Funções realizam cadastro, listagem dos títulos com exemplares disponíveis, busca pelo título e agregação para o gráfico Matplotlib. Os nomes de livros e autores usados na demonstração são dados fictícios; os resultados e testes são reais.

Indicador	Resultado
Títulos cadastrados	6
Títulos disponíveis	5
Exemplares disponíveis	15
Tecnologia	7
Ficção	7
História	1



O gráfico soma exemplares por gênero, não apenas títulos. Livro esgotado tem quantidade zero e permanece cadastrado, mas não aparece na lista de disponíveis. A busca ignora diferenças entre maiúsculas e minúsculas e espaços nas extremidades. Busca sem correspondência retorna uma lista vazia.

Verificações e avaliação dos resultados

Foram conferidos seis títulos cadastrados, cinco disponíveis, as somas por gênero e a busca por título com variação de caixa. Também foram testados título inexistente, título vazio, quantidade negativa e quantidade fracionária. As entradas inválidas foram rejeitadas sem inclusão na lista.

Por que uma lista de dicionários é uma boa escolha?

Uma lista reúne vários registros e permite percorrê-los, acrescentar elementos e pesquisar. Cada dicionário associa nomes claros aos campos, como título, autor e ano, tornando a leitura mais compreensível do que posições numéricas. Os registros podem ser convertidos em JSON ou em um DataFrame. É necessário validar os campos para manter consistência. Neste exercício, a classe Livro fornece uma estrutura comum para os objetos; a pergunta sobre listas de dicionários aborda outra representação possível para o cenário de título, autor e ano.

Limite do sistema

Os cadastros ficam em memória durante a execução. O JSON grava o resumo da demonstração, e não uma base completa de livros. Para conservar novos cadastros entre sessões, seria necessário adicionar persistência dos registros, por exemplo em SQLite.

Referências

Roteiro de aula prática U2 A4, material fornecido. Documento fornecido para a atividade, s.d.

Python, dataclasses. Disponível em: https://docs.python.org/3/library/dataclasses.html. Acesso em: 13 out. 2026.

Matplotlib, gráfico de barras. Disponível em: https://matplotlib.org/stable/plot_types/basic/bar.html. Acesso em: 13 out. 2026.

Codificação completa

Arquivo biblioteca.py. Inclui os comentários, as verificações e a gravação dos resultados.

```
# %% Importações e classe Livro
from collections import defaultdict
from dataclasses import dataclass
import json
import matplotlib.pyplot as plt

@dataclass
class Livro:
    titulo: str
    autor: str
    genero: str
    quantidade: int

# A lista começa vazia, como solicitado no roteiro.
livros = []

# %% Funções de cadastro, listagem e busca
def cadastrar_livro(titulo, autor, genero, quantidade):
    campos = [titulo.strip(), autor.strip(), genero.strip()]
    if not all(campos):
        raise ValueError('Título, autor e gênero são obrigatórios.')
    if type(quantidade) is not int or quantidade < 0:
        raise ValueError('Quantidade deve ser um inteiro não negativo.')
    livro = Livro(*campos, quantidade)
    livros.append(livro)
    return livro

def listar_livros():
    # Livros sem exemplares disponíveis permanecem no cadastro.
    return [livro for livro in livros if livro.quantidade > 0]

def buscar_livro(titulo):
    termo = titulo.strip().casefold()
    return [livro for livro in livros if livro.titulo.casefold() == termo]

def gerar_grafico():
    totais = defaultdict(int)
    for livro in livros:
        totais[livro.genero] += livro.quantidade
    fig, ax = plt.subplots(figsize=(8, 4))
    barras = ax.bar(list(totais), list(totais.values()), color='#2878a0')
    ax.bar_label(barras)
    ax.set(title='Exemplares disponíveis por gênero', ylabel='Exemplares')
    ax.set_ylim(0, max(totais.values(), default=0) + 2)
    fig.tight_layout()
    fig.savefig('livros_por_genero.png', dpi=180)
    plt.show()
    return dict(totais)

# %% Demonstração com dados fictícios de cadastro
cadastrar_livro('Python para iniciantes', 'Ana Souza', 'Tecnologia', 4)
cadastrar_livro('Algoritmos em Python', 'Bruno Lima', 'Tecnologia', 3)
cadastrar_livro('O jardim das estrelas', 'Carla Santos', 'Ficção', 5)
cadastrar_livro('Viagem ao futuro', 'Diego Alves', 'Ficção', 2)
```

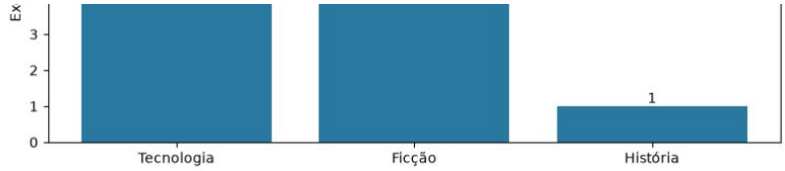
Codificação completa continuação

```
cadastrar_livro('História da cidade', 'Eva Rocha', 'História', 1)
cadastrar_livro('Livro esgotado', 'Fábio Costa', 'História', 0)
print('LIVROS DISPONÍVEIS')
for livro in listar_livros():
    print(f'{livro.titulo} | {livro.autor} | {livro.genero} | {livro.quantidade}')
print('\nBUSCA POR TÍTULO')
print(buscar_livro(' PYTHON PARA INICIANTES '))
print('Busca sem resultado:', buscar_livro('Título inexistente'))
totais = gerar_grafico()
print('Exemplares por gênero:', totais)

# %% Verificações de funcionamento
assert len(livros) == 6 and len(listar_livros()) == 5
assert buscar_livro('python para iniciantes')[0].quantidade == 4
assert buscar_livro('inexistente') == []
for quantidade in [-1, 1.5]:
    try:
        cadastrar_livro('Teste', 'Autor', 'Teste', quantidade)
    except ValueError:
        pass
    else:
        raise AssertionError('Quantidade inválida aceita.')
try:
    cadastrar_livro('', 'Autor', 'Teste', 1)
except ValueError:
    pass
else:
    raise AssertionError('Título vazio aceito.')
assert totais == {'Tecnologia': 7, 'Ficção': 7, 'História': 1}
print('TESTES: cadastro, busca, disponibilidade e validação aprovados.')
with open('resultados.json', 'w', encoding='utf-8') as arquivo:
    json.dump({'titulos': len(livros), 'disponiveis': len(listar_livros()),
               'exemplares': sum(totais.values()), 'generos': totais}, arquivo,
              ensure_ascii=False, indent=2)
```

Registro de execução

Captura do notebook com código e saída efetivamente executados no ambiente local. As demais células e saídas estão preservadas no arquivo .ipynb.




```
Exemplares por gênero: {'Tecnologia': 7, 'Ficção': 7, 'História': 1}
```

Verificações de funcionamento

In [4]:

```python
assert len(livros) == 6 and len(listar_livros()) == 5
assert buscar_livro('python para iniciantes')[0].quantidade == 4
assert buscar_livro('inexistente') == []
for quantidade in [-1, 1.5]:
    try:
        cadastrar_livro('Teste', 'Autor', 'Teste', quantidade)
    except ValueError:
        pass
    else:
        raise AssertionError('Quantidade inválida aceita.')
try:
    cadastrar_livro('', 'Autor', 'Teste', 1)
except ValueError:
    pass
else:
    raise AssertionError('Título vazio aceito.')
assert totais == {'Tecnologia': 7, 'Ficção': 7, 'História': 1}
print('TESTES: cadastro, busca, disponibilidade e validação aprovados.')
with open('resultados.json', 'w', encoding='utf-8') as arquivo:
    json.dump({'titulos': len(livros), 'disponiveis': len(listar_livros()),
               'exemplares': sum(totais.values()), 'generos': totais}, arquivo,
              ensure_ascii=False, indent=2)
```

```
TESTES: cadastro, busca, disponibilidade e validação aprovados.
```